Deploying an OpenClaw Agent as an Administrative Assistant in Microsoft 365

Executive summary

The selected MVP design is an **event-driven, app-only service** in Microsoft 365 that uses Microsoft Graph change notifications plus delta queries, runs under **OAuth 2.0 client credentials**, and is **mailbox-scoped with Exchange Online Application RBAC (RBAC for Applications)**. The service should monitor the approved mailbox set, classify scheduling traffic deterministically, read the contents of relevant existing meetings, compute availability, and reply through a dedicated assistant mailbox. There is no delegated-permissions fallback in this plan.

The assistant should be **meeting-context first**, not free/busy first. Before it proposes or changes time, it should read the actual calendar event data for relevant meetings unless the meeting is marked `sensitivity=private` or private-item policy prevents content access. The agent should normalize and reason over:

- logical organizer
- actual sender of the meeting message when available
- invited attendees (required, optional, resource)
- series metadata (`iCalUId`, `seriesMasterId`, recurrence)
- online-meeting metadata
- categories and subject/body cues
- whether the meeting looks dependency-bearing for other meetings or operating processes

Availability still matters, but it should be used **after** the assistant understands what the existing meetings are. `calendar/getSchedule` remains the right deterministic API for free/busy, but it is not the source of meeting meaning.

For outbound mail, the selected user-visible behavior is **Send on behalf**. The assistant mailbox should be the actual sender identity, and recipients should be able to see that it acted on behalf of the executive or mailbox owner. That is the better operational choice than Send As because it is transparent to recipients, easier to explain to IT, and cleaner for audit and trust.

The recommended hosting model remains a queue-backed webhook architecture: Graph subscriptions wake a thin webhook, the webhook enqueues work, and a deterministic worker performs message hydration, event reading, triage, scheduling, and reply generation.

MVP functional design and deterministic operating model

Functional contract

The MVP should implement the following contract:

1. Detect inbound scheduling activity from the mailbox.

2. Hydrate the scheduling context from **calendar events**, not just from email text.
3. Skip semantic reading for meetings marked private; treat those as busy blocks.
4. Apply deterministic rules to decide whether the item is protected, requires human approval, or can be auto-coordinated.
5. Read mailbox settings and free/busy only after the meeting context is classified.
6. Send the response through the dedicated assistant mailbox with the selected **Send on behalf** outcome.

Input classes

The assistant should handle two input classes:

- **Formal meeting traffic**: meeting requests, responses, and cancellations that appear as `eventMessage`, `eventMessageRequest`, or `eventMessageResponse`.
- **Ordinary scheduling mail**: regular email threads such as "can we move this?", "what works next week?", or "please find time with finance and legal."

For formal meeting traffic, the worker should expand the associated `event` directly from the message. For ordinary scheduling mail, the worker should search `calendarView` over a bounded time window and match candidate events using subject overlap, participant overlap, and timing clues.

Meeting-context-first model

For every candidate item, the worker should build a normalized meeting context object. At minimum, it should contain:

- mailbox owner
- message ID and conversation ID
- event ID if present
- message sender and logical `from`
- organizer
- required / optional / resource attendees
- start / end / time zone
- `iCalUId`, `seriesMasterId`, recurrence flag
- categories
- `isOrganizer`
- `allowNewTimeProposals`
- `isOnlineMeeting`
- sensitivity
- normalized subject and body text

This is how the agent answers the business questions that matter:

- **Who scheduled it?**
  Use the `eventMessage.sender` when the meeting came from mail, because that captures the actual submitting mailbox. Use `event.organizer` to identify the logical owner of the meeting. If a delegate

scheduled on behalf of the owner, `isOrganizer=true` can still apply to the owner, so you should preserve both values when they are available.

- **Who is included?**
  Use the event attendee collection, not just message recipients. Preserve attendee type (`required`, `optional`, `resource`) and current response status.
- **Is this meeting dependency-bearing?**
  Determine that with deterministic rules over metadata and an explicit policy table that you control. Good signals are recurring-series membership, protected categories, VIP organizers, large attendee sets, external customer domains, room/resource attendees, subject/body keywords, and known protected meeting series. Do not leave this inference entirely to the model.

Private-meeting rule

Private meetings should be handled explicitly:

- If `sensitivity=private`, or private-item access is intentionally not granted, the assistant should not ingest the body, subject, or attendee semantics of that item.
- It should still mark the time as unavailable.
- It should log that the block was treated as **busy-only**.

That preserves the user's privacy boundary while still allowing deterministic scheduling.

Dependency model recommendation

The best deterministic setup is a **two-layer dependency model**:

1. **Static policy rules you maintain**
   Example: subject patterns such as `Board`, `SteerCo`, `QBR`, `Launch Review`, `Interview Loop`, `1:1`, `Finance Close`; protected categories such as `Executive`, `Customer`, `Launch`, `Hiring`; protected organizers such as the CEO, CFO, or Chief of Staff; and critical attendee sets such as direct reports or customer contacts.
2. **Computed event signals**
   Example: recurring meetings, large attendee counts, resource/room bookings, external attendees, online-meeting flags, recent modifications, and event/message linkage showing someone is proposing a change.

The model may use an LLM to summarize the meeting text, but the **decision class** must come from deterministic code.

Microsoft Graph APIs and endpoints required

Mail ingestion and meeting-context hydration

Use these APIs in the MVP:

- `GET /users/{id}/mailFolders/{id}/messages/delta`
  Incremental sync for inbox changes. Use this as the durable reconciliation layer.

- `GET /users/{id}/messages/{message-id}`
  Read the candidate message with a bounded `$select`.
- `GET /users/{id}/messages/{message-id}?$expand=microsoft.graph.eventMessage/event(...)`
  For meeting-related mail, expand the associated calendar event directly from the message so the worker can capture both the actual sender and the event metadata.
- `GET /users/{id}/calendarView?startDateTime=...&endDateTime=...`
  Read the actual meetings in a bounded time window. This should be the default event-content read surface because it returns occurrences, exceptions, and single instances. Send:
  - `Prefer: outlook.timezone="..."` for deterministic time rendering
  - `Prefer: outlook.body-content-type="text"` so downstream logic receives event body text instead of HTML
- `GET /users/{id}/events/{event-id}`
  Hydrate a specific event by ID when the worker already knows the event to inspect. Normalize the returned HTML body in code if you use this route.

Availability and scheduling context

Use these APIs after the meeting has been classified:

- `GET /users/{id}/mailboxSettings`
  Read time zone and working hours.
- `POST /users/{id}/calendar/getSchedule`
  Compute deterministic free/busy for the mailbox owner and any required participants. This API should be used for slot computation, not for understanding meeting meaning.

Outbound response path

Use these APIs for the selected mail behavior:

- `POST /users/{assistantMailboxUpn}/sendMail`
  Send replies from the dedicated assistant mailbox.
- For the selected **Send on behalf** outcome, set the message `from` to the principal mailbox and submit the message through the assistant mailbox. The tenant should be configured so the resulting message renders as "Assistant on behalf of Executive."

Later write path for calendar changes

When you are ready to enable calendar writes:

- `PATCH /users/{id}/events/{event-id}`
  Reschedule meetings when the mailbox owner is the organizer.
- `POST /users/{id}/events/{event-id}/tentativelyAccept` with `proposedNewTime`
  Use the attendee-side propose-new-time workflow when the mailbox owner is not the organizer and the event allows proposals.

Identity, OAuth flow, permissions, and token management

Selected auth model

Use **only** this auth model for the MVP:

- **OAuth 2.0 client credentials**
- **Application permissions**
- **Exchange Online Application RBAC** to scope the app to the allowed mailbox set

There is no delegated path in the implementation plan.

Required application roles

For the read-and-reply MVP, the minimum Exchange application role set should be:

- `Application Mail.Read`
- `Application Calendars.Read`
- `Application MailboxSettings.Read`
- `Application Mail.Send`

Add this only when calendar writes are enabled:

- `Application Calendars.ReadWrite`

Do not use `Application Exchange Full Access` or similarly broad roles for this MVP.

Critical scoping rule

Application RBAC is mailbox-scoped in Exchange Online, but **unscoped Microsoft Entra application grants are additive**. If the app still has broad Graph application permissions consented tenant-wide outside the intended Exchange scope, those broad grants can undermine the scoping model. The administrator should explicitly review and remove overlapping broad grants before pilot rollout.

Token handling

Use a confidential client with certificate-based authentication where possible. Store the certificate or secret in Key Vault or the equivalent secret manager, cache tokens in-process, and fail closed when the credential is missing or expired.

Mailbox and calendar delegation design

Why Send on behalf is the selected route

There are two recipient-visible outcomes when mail appears to originate from another mailbox:

- **Send As**: the message looks as though it came directly from the principal mailbox.
- **Send on behalf**: recipients can see that another mailbox acted on behalf of the principal mailbox.

For this assistant, **Send on behalf is the better route** because it preserves transparency. Recipients can see that an assistant service acted, which is better for trust, change management, and audit review. Do not optimize for illusion; optimize for operational clarity.

Selected configuration

Use a **dedicated assistant mailbox** as the operational sender. Then configure the assistant mailbox so it can send **on behalf of** the principal mailbox.

Recommended Exchange configuration for each principal mailbox:

```
# Allow the assistant mailbox to open the principal mailbox when needed
Add-MailboxPermission `
  -Identity "executive@contoso.com" `
  -User "assistant@contoso.com" `
  -AccessRights FullAccess `
  -InheritanceType All `
  -AutoMapping $false

# Grant Send on behalf to the assistant mailbox
Set-Mailbox `
  -Identity "executive@contoso.com" `
  -GrantSendOnBehalfTo @{Add="assistant@contoso.com"}
```

Operational rules:

- Put the assistant mailbox **inside the Application RBAC scope**.
- Grant `Application Mail.Send` to the app within that scope.
- Do **not** grant Send As for this workflow. If Send As is also present, Exchange can prefer the less transparent outcome.
- Maintain an explicit allowlist of principal mailboxes that the assistant mailbox is allowed to represent.

Selected send behavior

The app should always submit mail through the assistant mailbox and set `from` to the principal mailbox.

Example request:

```
POST https://graph.microsoft.com/v1.0/users/assistant@contoso.com/sendMail
Content-Type: application/json

{
  "message": {
    "subject": "Re: QBR scheduling",
    "from": {
      "emailAddress": {
        "address": "executive@contoso.com"
      }
    },
    "toRecipients": [
      {
        "emailAddress": {
          "address": "customer@fabrikam.com"
        }
```

```
      }
    ],
    "body": {
      "contentType": "Text",
      "content": "I'm writing on behalf of Executive Name. Here are three times that
work next week..."
    }
  },
  "saveToSentItems": true
}
```

Before broad rollout, validate the exact rendered result in Outlook and OWA for your tenant and confirm it appears as the intended **Send on behalf** experience.

Calendar-write behavior

Keep calendar writes behind feature flags:

- `ENABLE_ORGANIZER_RESCHEDULE=true` for organizer-owned `PATCH` updates
- `ENABLE_ATTENDEE_PROPOSE_NEW_TIME=true` for attendee-side proposal flows

Do not silently rewrite someone else's meeting when the mailbox owner is not the organizer.

Eventing, synchronization, and error handling

Subscriptions and webhook design

Use Graph subscriptions for mailbox activity and keep the webhook thin:

1. Validate the subscription handshake quickly.
2. Enqueue a work item.
3. Return immediately.
4. Do all Graph reads, classification, and reply generation in the worker.

Track subscription IDs, expiration timestamps, and client state in durable storage. Renew subscriptions on a schedule.

Delta as the source of truth

Treat webhooks as wake signals and `messages/delta` as the authoritative reconciliation mechanism. The worker should preserve `@odata.deltaLink` per mailbox and periodically reconcile to recover from missed, duplicated, or delayed notifications.

Retry and idempotency

Implement:

- `429` handling using `Retry-After`
- exponential backoff when `Retry-After` is absent
- queue retry limits and dead-lettering
- idempotency keys for mail sends and calendar writes

For mail, use an internal dedupe key such as:

`{tenantId}:{mailboxUpn}:{messageId}:{actionType}`

For calendar writes, also persist the target `eventId` and a transaction or correlation ID.

Security and compliance considerations

Least privilege and blast radius

This service reads mail and calendar data and can send mail externally. Keep the blast radius bounded:

- scope mailbox access with Exchange Application RBAC
- keep the mailbox scope explicit and reviewable
- avoid broad application roles
- allow only specific sender mailboxes
- keep write features behind flags until auditing is in place

Audit and monitoring

Enable:

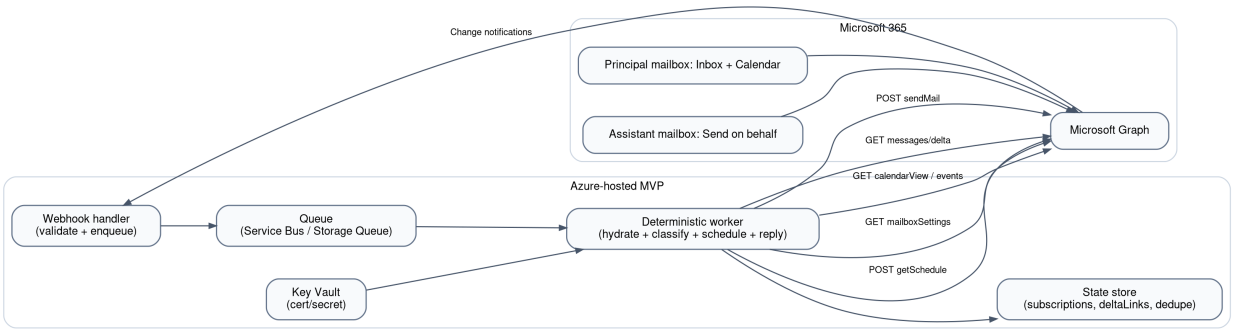
- Microsoft Graph activity logging
- Microsoft Purview audit review for delegation changes and agent actions
- structured application logs with mailbox, event, message, correlation ID, and action outcome

OpenClaw runtime hardening

Treat email bodies, attachments, and event text as untrusted input. The model may extract constraints, but tool execution must be guarded by deterministic code and explicit policy checks. Keep the skill/plugin surface tightly allowlisted.

Deployment options, reference architecture, and MVP plan

Recommended architecture



Deterministic triage rule: recommended implementation

The triage rule should be **action-oriented**, not score-oriented. Use five deterministic outputs:

- `IGNORE`
- `PRIVATE_BUSY_ONLY`
- `PROTECTED_MEETING`
- `HUMAN_APPROVAL`
- `AUTO_COORDINATE`

That maps much more cleanly to product behavior than vague priority integers.

Concrete TypeScript implementation

The following code is designed for a Node.js / Azure Functions worker and uses direct Graph REST calls. It reads the message, expands the associated event when available, falls back to `calendarView` when needed, skips private-item semantics, and classifies the item deterministically.

```
type Decision =
  | "IGNORE"
  | "PRIVATE_BUSY_ONLY"
  | "PROTECTED_MEETING"
  | "HUMAN_APPROVAL"
  | "AUTO_COORDINATE";

type GraphAddress = {
  emailAddress?: {
    name?: string | null;
    address?: string | null;
  };
};

type GraphAttendee = GraphAddress & {
  type?: "required" | "optional" | "resource";
  status?: {
    response?: string;
  };
};

type GraphEvent = {
  id?: string;
  iCalUId?: string;
  seriesMasterId?: string;
  subject?: string;
  bodyPreview?: string;
  body?: { content?: string; contentType?: string };
  organizer?: GraphAddress;
  attendees?: GraphAttendee[];
  categories?: string[];
  isOrganizer?: boolean;
  isOnlineMeeting?: boolean;
  allowNewTimeProposals?: boolean;
  sensitivity?: string;
  start?: { dateTime?: string; timeZone?: string };
```

```
  end?: { dateTime?: string; timeZone?: string };
  lastModifiedDateTime?: string;
  type?: string;
};

type GraphMessage = {
  id?: string;
  subject?: string;
  bodyPreview?: string;
  body?: { content?: string; contentType?: string };
  from?: GraphAddress;
  sender?: GraphAddress;
  toRecipients?: GraphAddress[];
  ccRecipients?: GraphAddress[];
  conversationId?: string;
  receivedDateTime?: string;
  meetingMessageType?: string;
  event?: GraphEvent;
};

type NormalizedMeetingContext = {
  mailboxUpn: string;
  messageId: string;
  conversationId: string;
  eventId?: string;
  subject: string;
  bodyText: string;
  messageSender: string;
  messageFrom: string;
  organizer: string;
  requiredAttendees: string[];
  optionalAttendees: string[];
  resourceAttendees: string[];
  allAttendees: string[];
  categories: string[];
  isMeetingMessage: boolean;
  isOrganizer: boolean;
  isRecurring: boolean;
  isOnlineMeeting: boolean;
  allowNewTimeProposals: boolean;
  sensitivity: string;
  iCalUId?: string;
  seriesMasterId?: string;
  receivedDateTime?: string;
  lastModifiedDateTime?: string;
};

type TriageResult = {
  decision: Decision;
  reasons: string[];
};

const CONFIG = {
  internalDomains: new Set(["contoso.com"]),
  vipOrganizers: new Set([
    "ceo@contoso.com",
    "chief.of.staff@contoso.com",
```

```
    "cfo@contoso.com",
  ]),
  protectedCategories: new Set([
    "Executive",
    "Board",
    "Customer",
    "Launch",
    "Hiring",
    "FinanceClose",
  ]),
  protectedSubjectPatterns: [
    /\b(board|steerco|steering committee|exec staff|staff meeting)\b/i,
    /\b(qbr|ebr|renewal|customer escalation|customer review)\b/i,
    /\b(launch review|go[- ]live|change advisory|cab|incident review)\b/i,
    /\b(interview loop|candidate debrief|performance review|finance close)\b/i,
    /\b(1:1|one on one)\b/i,
  ],
  largeMeetingThreshold: 6,
};

function normalizeEmail(value?: string | null): string {
  return (value ?? "").trim().toLowerCase();
}

function emailOf(recipient?: GraphAddress): string {
  return normalizeEmail(recipient?.emailAddress?.address);
}

function stripHtml(value?: string | null): string {
  return (value ?? "")
    .replace(/<style[\s\S]*?<\/style>/gi, " ")
    .replace(/<script[\s\S]*?<\/script>/gi, " ")
    .replace(/<[^>]+>/g, " ")
    .replace(/ /gi, " ")
    .replace(/&amp;/gi, "&")
    .replace(/\s+/g, " ")
    .trim();
}

function isInternal(email: string): boolean {
  const domain = email.split("@")[1] ?? "";
  return CONFIG.internalDomains.has(domain);
}

function normalizeAttendees(
  attendees: GraphAttendee[] = []
): Pick<
  NormalizedMeetingContext,
  "requiredAttendees" | "optionalAttendees" | "resourceAttendees" | "allAttendees"
> {
  const required: string[] = [];
  const optional: string[] = [];
  const resource: string[] = [];

  for (const attendee of attendees) {
    const email = emailOf(attendee);
    if (!email) continue;
```

```
    if (attendee.type === "optional") optional.push(email);
    else if (attendee.type === "resource") resource.push(email);
    else required.push(email);
  }

  return {
    requiredAttendees: required,
    optionalAttendees: optional,
    resourceAttendees: resource,
    allAttendees: [...required, ...optional, ...resource],
  };
}

function normalizeContext(
  mailboxUpn: string,
  message: GraphMessage,
  event?: GraphEvent
): NormalizedMeetingContext {
  const attendees = normalizeAttendees(event?.attendees ?? []);
  const bodyText =
    event?.body?.contentType?.toLowerCase() === "html"
      ? stripHtml(event.body.content)
      : event?.body?.content?.trim() ||
        message.bodyPreview?.trim() ||
        stripHtml(message.body?.content);

  return {
    mailboxUpn,
    messageId: message.id ?? "",
    conversationId: message.conversationId ?? "",
    eventId: event?.id,
    subject: (event?.subject ?? message.subject ?? "").trim(),
    bodyText: bodyText ?? "",
    messageSender: emailOf(message.sender),
    messageFrom: emailOf(message.from),
    organizer: emailOf(event?.organizer),
    categories: (event?.categories ?? []).map((x) => x.trim()).filter(Boolean),
    isMeetingMessage: Boolean(message.meetingMessageType),
    isOrganizer: Boolean(event?.isOrganizer),
    isRecurring: Boolean(event?.seriesMasterId || event?.type === "seriesMaster"),
    isOnlineMeeting: Boolean(event?.isOnlineMeeting),
    allowNewTimeProposals: Boolean(event?.allowNewTimeProposals),
    sensitivity: (event?.sensitivity ?? "normal").toLowerCase(),
    iCalUId: event?.iCalUId,
    seriesMasterId: event?.seriesMasterId,
    receivedDateTime: message.receivedDateTime,
    lastModifiedDateTime: event?.lastModifiedDateTime,
    ...attendees,
  };
}

function dependencyScore(ctx: NormalizedMeetingContext): number {
  let score = 0;
  const searchableText = `${ctx.subject} ${ctx.bodyText}`;

  if (ctx.isRecurring) score += 2;
  if (ctx.allAttendees.length >= CONFIG.largeMeetingThreshold) score += 2;
```

```
  if (ctx.resourceAttendees.length > 0) score += 1;
  if (ctx.isOnlineMeeting) score += 1;
  if (ctx.categories.some((c) => CONFIG.protectedCategories.has(c))) score += 3;
  if (CONFIG.protectedSubjectPatterns.some((rx) => rx.test(searchableText))) score += 3;
  if (CONFIG.vipOrganizers.has(ctx.organizer)) score += 3;
  if (ctx.allAttendees.some((email) => !isInternal(email))) score += 2;

  return score;
}

export function triage(ctx: NormalizedMeetingContext): TriageResult {
  if (!ctx.subject && !ctx.bodyText) {
    return { decision: "IGNORE", reasons: ["No usable scheduling content"] };
  }

  if (ctx.sensitivity === "private") {
    return {
      decision: "PRIVATE_BUSY_ONLY",
      reasons: ["Event is marked private; treat as unavailable but do not ingest
semantics"],
    };
  }

  const depScore = dependencyScore(ctx);
  const organizerIsVip = CONFIG.vipOrganizers.has(ctx.organizer);
  const senderIsInternal = isInternal(ctx.messageSender || ctx.messageFrom);

  if (organizerIsVip || depScore >= 7) {
    return {
      decision: "PROTECTED_MEETING",
      reasons: [
        organizerIsVip ? "Protected organizer" : "High dependency score",
        `dependencyScore=${depScore}`,
      ],
    };
  }

  if (!senderIsInternal || depScore >= 4) {
    return {
      decision: "HUMAN_APPROVAL",
      reasons: [
        !senderIsInternal ? "External sender or participant present" : "Moderate
dependency score",
        `dependencyScore=${depScore}`,
      ],
    };
  }

  return {
    decision: "AUTO_COORDINATE",
    reasons: ["Internal, non-private, non-protected meeting with low dependency score"],
  };
}

async function graphGet<T>(
  token: string,
  url: string,
```

```
  extraHeaders: Record<string, string> = {}
): Promise<T> {
  const response = await fetch(url, {
    method: "GET",
    headers: {
      Authorization: `Bearer ${token}`,
      Accept: "application/json",
      ...extraHeaders,
    },
  });

  if (!response.ok) {
    throw new Error(`Graph GET failed: ${response.status} ${await response.text()}`);
  }

  return (await response.json()) as T;
}

function chooseMostLikelyRelatedEvent(message: GraphMessage, events: GraphEvent[]):
GraphEvent | undefined {
  const messageTokens = new Set(
    (message.subject ?? "")
      .toLowerCase()
      .split(/[^a-z0-9]+/)
      .filter((x) => x.length >= 4)
  );

  const people = new Set([
    emailOf(message.from),
    emailOf(message.sender),
    ...(message.toRecipients ?? []).map(emailOf),
    ...(message.ccRecipients ?? []).map(emailOf),
  ]);

  let best: { event?: GraphEvent; score: number } = { score: 0 };

  for (const event of events) {
    const subjectTokens = new Set(
      (event.subject ?? "")
        .toLowerCase()
        .split(/[^a-z0-9]+/)
        .filter((x) => x.length >= 4)
    );

    const attendeeEmails = new Set((event.attendees ??
[]).map(emailOf).filter(Boolean));
    let score = 0;

    for (const token of messageTokens) {
      if (subjectTokens.has(token)) score += 2;
    }

    for (const email of people) {
      if (email && attendeeEmails.has(email)) score += 3;
    }

    if (score > best.score) best = { event, score };
```

```
  }

  return best.score >= 4 ? best.event : undefined;
}

export async function hydrateAndTriage(
  token: string,
  mailboxUpn: string,
  messageId: string
): Promise<{ context: NormalizedMeetingContext; result: TriageResult }> {
  const messageUrl =
    `https://graph.microsoft.com/v1.0/users/${encodeURIComponent(mailboxUpn)}` +
    `/messages/${encodeURIComponent(messageId)}` +
    `?
$select=id,subject,bodyPreview,body,from,sender,toRecipients,ccRecipients,conversationId,receivedDateT
+
    `&$expand=microsoft.graph.eventMessage/event(` +

`$select=id,iCalUId,seriesMasterId,subject,bodyPreview,body,organizer,attendees,categories,isOrganizer
+
    `)`;

  const message = await graphGet<GraphMessage>(token, messageUrl);
  let event = message.event;

  if (!event) {
    const now = new Date();
    const end = new Date(now.getTime() + 14 * 24 * 60 * 60 * 1000);

    const calendarViewUrl =
      `https://graph.microsoft.com/v1.0/users/${encodeURIComponent(mailboxUpn)}` +
      `/calendarView?startDateTime=${encodeURIComponent(now.toISOString())}` +
      `&endDateTime=${encodeURIComponent(end.toISOString())}`;

    const calendarView = await graphGet<{ value: GraphEvent[] }>(token, calendarViewUrl,
{
      Prefer: 'outlook.timezone="UTC", outlook.body-content-type="text"',
    });

    event = chooseMostLikelyRelatedEvent(message, calendarView.value ?? []);
  }

  const context = normalizeContext(mailboxUpn, message, event);
  const result = triage(context);
  return { context, result };
}
```

How the Graph API should be called

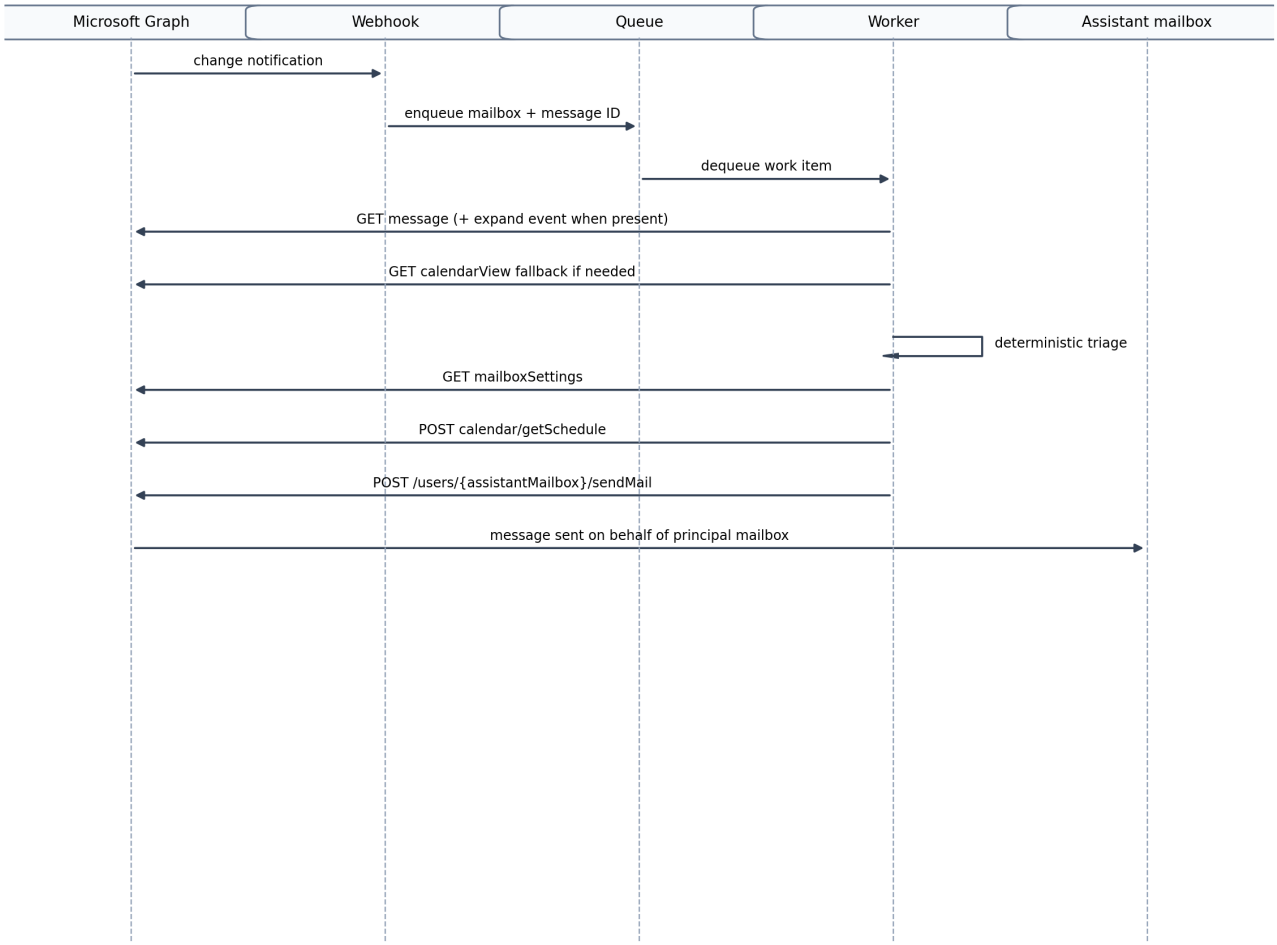
Use the code above in this sequence:

1. **Wake up on mailbox changes**
   - `GET /users/{mailbox}/mailFolders/Inbox/messages/delta?changeType=created`
   - or process the mailbox item IDs delivered by the subscription pipeline.

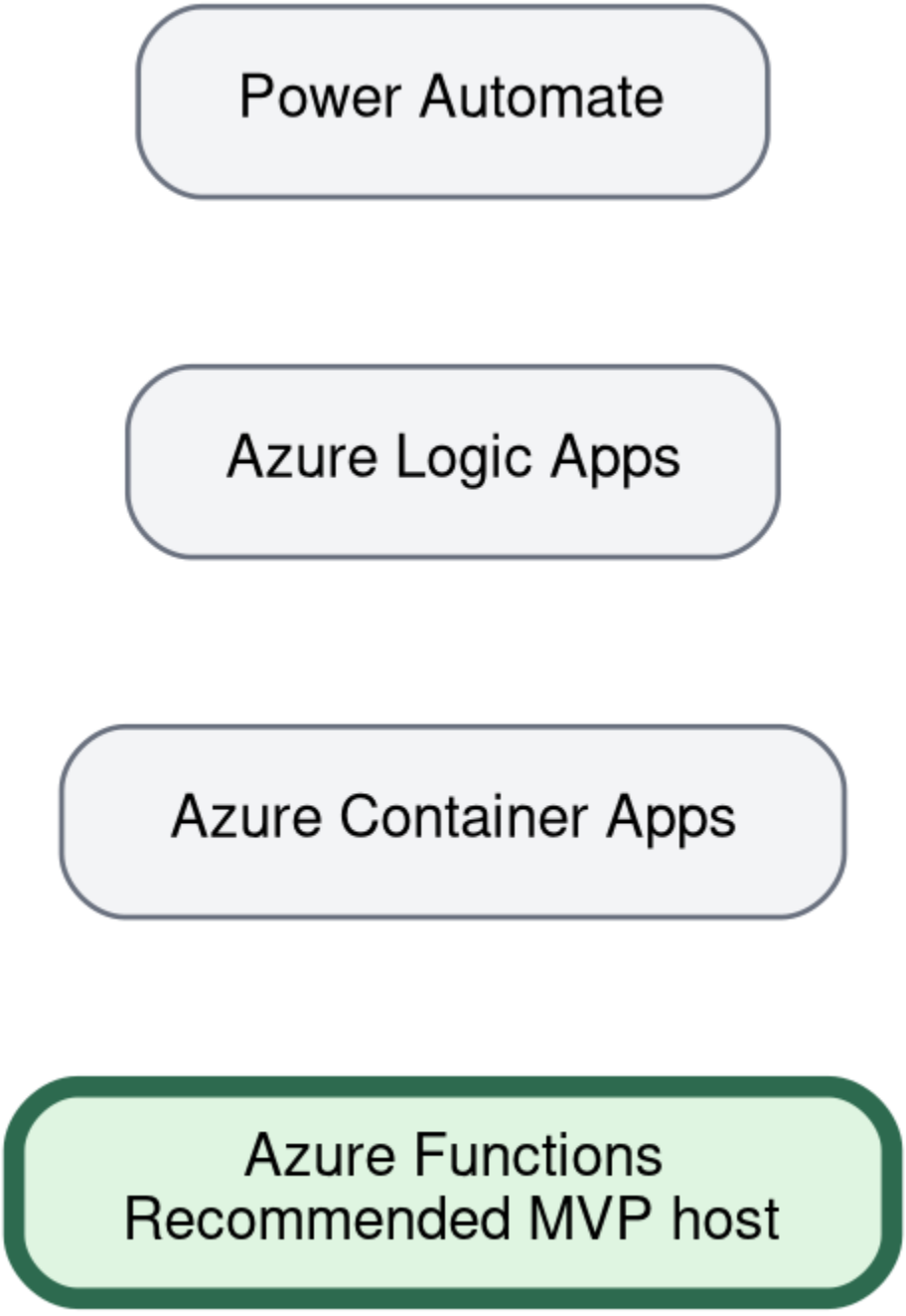
2. **Hydrate the scheduling object**
    - `GET /users/{mailbox}/messages/{messageId}?$expand=microsoft.graph.eventMessage/event(...)`
    - If the message is a meeting request, that single call gives you both:
        - the actual message sender
        - the associated calendar event
3. **Fallback to calendar context for ordinary scheduling email**
    - `GET /users/{mailbox}/calendarView?...`
    - Use `Prefer: outlook.body-content-type="text"` so the event body is directly usable in deterministic code.
4. **Apply the deterministic triage rule**
    - If the event is private, return `PRIVATE_BUSY_ONLY`.
    - Otherwise classify the item into `PROTECTED_MEETING`, `HUMAN_APPROVAL`, or `AUTO_COORDINATE`.
5. **Only then read scheduling inputs**
    - `GET /users/{mailbox}/mailboxSettings`
    - `POST /users/{mailbox}/calendar/getSchedule`
6. **Send the reply through the assistant mailbox**
    - `POST /users/{assistantMailbox}/sendMail`

That sequence makes the assistant dependency-aware before it starts proposing time.

End-to-end flow example



Implementation options comparison



Option	Strengths	Constraints / risks	Best fit for MVP
	Best match for Graph webhooks + queue workers; easy to separate the		

Option	Strengths	Constraints / risks	Best fit for MVP
✅ **Azure Functions (recommended)**	validation endpoint from deterministic processing; lowest friction for a hardened event-driven MVP	Cold start and runtime packaging must be managed	**Best overall fit for the selected design**
Azure Container Apps	Strong option when the worker or OpenClaw runtime is already containerized; good for longer-running jobs	Slightly more operational surface area than Functions	Good second choice
Azure Logic Apps	Useful as an orchestrator around code-based components	Deterministic policy logic becomes harder to maintain as rules grow	Supplementary, not primary
Power Automate	Fastest prototype path for simple automation	Too little control for Application RBAC, deterministic policy, and mailbox-scoped assistant behavior	Not recommended for this MVP

Prioritized step-by-step MVP plan with effort and risks

Foundation and tenant setup (2–4 days)

- Register the Entra app as **single-tenant** for the MVP.
- Create the Exchange service-principal reference with `New-ServicePrincipal`.
- Create the mailbox scope using either:
  - an Exchange Management Scope, or
  - an Administrative Unit
- Assign these Exchange application roles to the app within that scope:
  - `Application Mail.Read`
  - `Application Calendars.Read`
  - `Application MailboxSettings.Read`
  - `Application Mail.Send`
- Create the dedicated assistant mailbox.
- Put the assistant mailbox **inside the same allowed mailbox scope**.
- Configure **Full Access + Send on behalf** from each principal mailbox to the assistant mailbox.
- Explicitly remove or avoid **Send As** for this workflow.
- Review and remove overlapping broad Entra Graph application grants that would bypass the intended Exchange scope.

Risks:

- admin consent and RBAC setup can take coordination time
- broad Entra grants can accidentally defeat the scoping design

- Send As lingering in the tenant can undercut the intended transparent behavior

Ingestion and event hydration (3–6 days)

- Build the webhook endpoint and queue pipeline.
- Create and renew inbox subscriptions.
- Implement `messages/delta`.
- Implement message read plus `eventMessage/event` expansion.
- Implement `calendarView` fallback for ordinary scheduling mail.

Risks:

- silent degradation if subscription renewal is missed
- overly broad hydration can increase cost and noise if not bounded by time windows and `$select`

Deterministic triage and scheduling (4–7 days)

- Implement the action-oriented triage rule shown above.
- Build the protected-meeting policy table.
- Implement mailbox settings reads.
- Implement `calendar/getSchedule`.
- Generate deterministic candidate slots.
- Send replies through the assistant mailbox with the selected Send on behalf behavior.

Risks:

- poor protected-meeting policy design can either over-block or over-automate
- time-zone and working-hours assumptions can drift if mailbox settings are ignored

Operational hardening (2–5 days)

- Add retry, dedupe, and dead-letter handling.
- Turn on Graph and Purview audit review.
- Add per-mailbox feature flags.
- Add global kill switches for outbound mail and calendar writes.
- Roll out to a small mailbox set first.

Risks:

- outbound actions without kill switches create unnecessary blast radius
- poor observability makes mailbox-scope mistakes harder to detect

Later calendar-write path (3–6 days)

- Add `Application Calendars.ReadWrite` only when approved.
- Enable organizer-owned rescheduling first.

- Add attendee-side propose-new-time second.
- Keep both flows behind feature flags.

Risks:

- mistaken reschedules are higher-stakes than mistaken replies
- attendee-side proposal behavior is easy to confuse with organizer-side event updates unless the product logic keeps them separate

Administrator checklist

1) Create the app and Exchange service-principal reference

Record and hand to engineering:

- Tenant ID
- Client / Application ID
- Enterprise Application service principal Object ID
- certificate or secret details
- expiration date for the credential

Create the Exchange Online service-principal pointer:

```
Connect-ExchangeOnline

New-ServicePrincipal `
  -AppId "<CLIENT_APPLICATION_ID>" `
  -ObjectId "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -DisplayName "OpenClaw Assistant"
```

2) Define the mailbox scope

Create either:

- an Exchange Management Scope, or
- an Administrative Unit-based recipient scope

Example Management Scope based on a scoping group:

```
Connect-ExchangeOnline

Get-Group -Identity "OpenClaw Scoped Mailboxes" | Format-List Name,DistinguishedName

New-ManagementScope `
  -Name "OpenClaw-ScopedMailboxes" `
  -RecipientRestrictionFilter "MemberOfGroup -eq '<GROUP_DISTINGUISHED_NAME>'"
```

3) Assign Application RBAC roles

```
# Mail read
New-ManagementRoleAssignment `
  -Name "OpenClaw-MailRead" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application Mail.Read" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"

# Calendar read
New-ManagementRoleAssignment `
  -Name "OpenClaw-CalendarsRead" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application Calendars.Read" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"

# Mailbox settings read
New-ManagementRoleAssignment `
  -Name "OpenClaw-MailboxSettingsRead" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application MailboxSettings.Read" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"

# Mail send
New-ManagementRoleAssignment `
  -Name "OpenClaw-MailSend" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application Mail.Send" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"
```

Add this later for calendar writes:

```
New-ManagementRoleAssignment `
  -Name "OpenClaw-CalendarsReadWrite" `
  -App "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Role "Application Calendars.ReadWrite" `
  -CustomResourceScope "OpenClaw-ScopedMailboxes"
```

If you use an Administrative Unit instead of a Management Scope, replace `-CustomResourceScope` with `-RecipientAdministrativeUnitScope`.

4) Configure the assistant mailbox for the selected send behavior

For each principal mailbox that the assistant represents:

```
# Assistant can access the principal mailbox when required
Add-MailboxPermission `
  -Identity "executive@contoso.com" `
  -User "assistant@contoso.com" `
  -AccessRights FullAccess `
  -InheritanceType All `
  -AutoMapping $false

# Assistant sends on behalf of the principal mailbox
Set-Mailbox `
```

```
  -Identity "executive@contoso.com" `
  -GrantSendOnBehalfTo @{Add="assistant@contoso.com"}
```

Administrative rules:

- keep the assistant mailbox in scope
- do not grant Send As for this workflow
- document exactly which principal mailboxes the assistant mailbox may represent

5) Review Entra app permissions

Explicitly review the Enterprise Application admin-consent list and remove broad overlapping Graph application permissions that would exceed the intended Exchange scope.

6) Validate the scope before development

Use one allowed mailbox and one disallowed mailbox:

```
Test-ServicePrincipalAuthorization `
  -Identity "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Resource "in-scope-user@contoso.com"

Test-ServicePrincipalAuthorization `
  -Identity "<ENTERPRISE_APPLICATION_SERVICE_PRINCIPAL_OBJECT_ID>" `
  -Resource "out-of-scope-user@contoso.com"
```

Give engineering:

- one in-scope mailbox
- one out-of-scope mailbox
- the assistant mailbox UPN
- confirmation that Send As is not configured for the selected workflow

Developer implementation checklist

1) Capture the admin handoff in configuration

At minimum:

- `TENANT_ID`
- `CLIENT_ID`
- certificate or secret reference
- `ASSISTANT_MAILBOX_UPN`
- `IN_SCOPE_TEST_MAILBOX`
- `OUT_OF_SCOPE_TEST_MAILBOX`
- feature flags for calendar writes

2) Implement app-only auth first

Before business logic:

- acquire a client-credentials token
- read an in-scope mailbox successfully
- fail against an out-of-scope mailbox
- log the result as part of startup validation

3) Build the ingestion pipeline

Implement:

- mailbox subscriptions
- webhook validation
- queue-backed workers
- message delta reconciliation

4) Build meeting hydration before scheduling logic

Implement this order:

1. read message
2. expand event from `eventMessage` when present
3. fallback to `calendarView`
4. normalize the meeting context
5. apply the deterministic triage rule
6. only then call mailbox settings and `getSchedule`

That ordering is important. It prevents the assistant from treating all meetings as interchangeable free/busy blocks.

5) Implement the selected send path

For every outbound reply:

- submit through `/users/{assistantMailboxUpn}/sendMail`
- set `from` to the principal mailbox
- log the assistant mailbox, principal mailbox, recipients, message correlation ID, and triage result
- reject any request that tries to use a mailbox outside the allowlist

6) Keep writes behind flags

Recommended rollout order:

1. read-only scheduling assistant
2. outbound replies
3. organizer-side calendar updates
4. attendee-side propose-new-time

7) Add kill switches and audit metadata

Before broader rollout, add:

- global kill switch for outbound sends
- global kill switch for calendar writes
- per-mailbox enablement flags
- idempotency keys
- structured logs for mailbox, event ID, old/new time, action, and correlation ID

Selected Microsoft Learn references

- Role Based Access Control for Applications in Exchange Online
- Send Outlook messages from another user
- event resource type
- Get eventMessage
- List calendarView
- Get user mailbox settings
- calendar/getSchedule
- message: delta
- Test-ServicePrincipalAuthorization
- New-ServicePrincipal